



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Платформенные решения для интеллектуального анализа больших данных»

КАФЕДРА «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе № 2

по курсу «Технология параллельных систем баз данных»

на тему: «Оптимизация запросов. Основы EXPLAIN в PostgreSQL. Индексация»

Вариант № 1

Студент ИУ6-13М
(Группа)

Преподаватель

(Подпись, дата)

(Подпись, дата)

Синдеев В. А.
(И. О. Фамилия)

Григорьев Ю. А.
(И. О. Фамилия)

2026 г.

1 Цель работы

Формирование следующей компетенции: студент должен получить навыки работы с командой EXPLAIN. Он также должен познакомиться с эффективными методами индексации в PostgreSQL

2 Основы EXPLAIN

Explain, Результат представлен на рисунке 1.

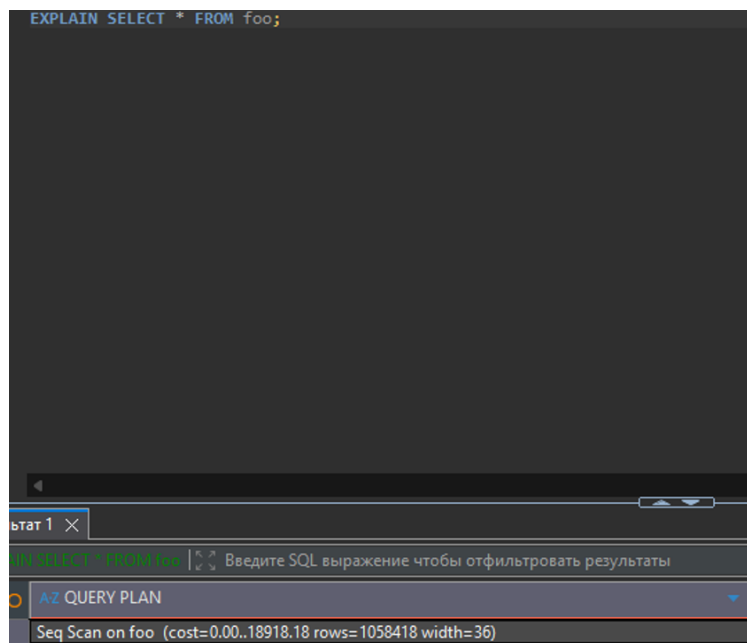


Рис. 1 – Результат запроса

Добавили 10 строк, Результат представлен на рисунке 2.

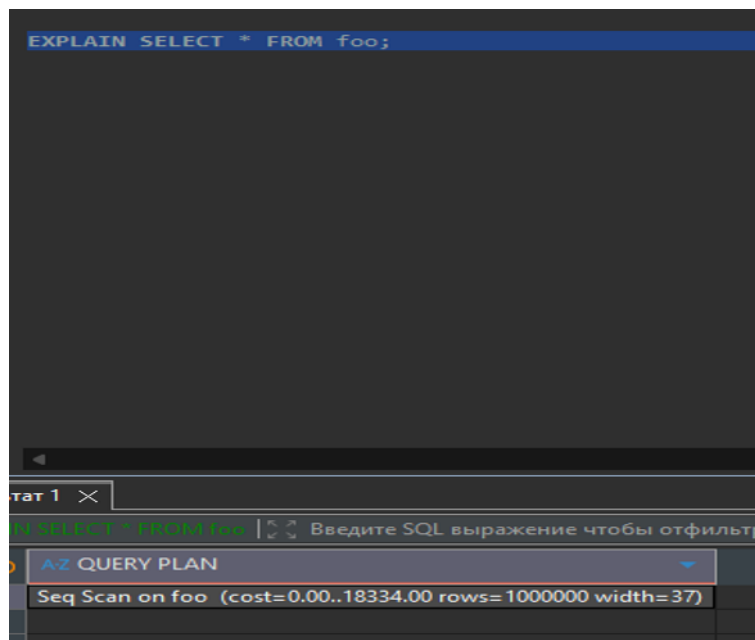


Рис. 2 – План выполнения запроса + 10 строка

Данные планировщика не обновились, из-за малого количества добавленных строк. Можно обновить вручную.

Выполнили команду `ANALYZE foo`, которая обновляет принудительно статистику для анализатора. Результат представлен на рисунке 3.

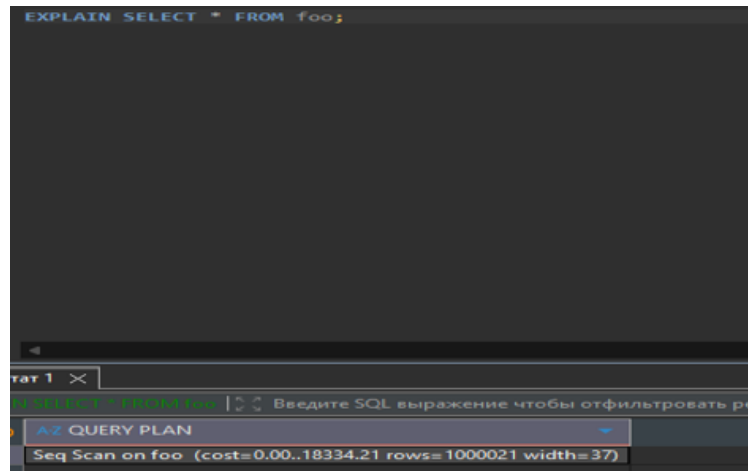


Рис. 3 – План выполнения запроса после обновления данных у планировщика

Запрос вместе с `analyze`, мы получаем дополнительную информацию Actual time реальное время полного прохождения по таблице Rows количество строк пройденных в узле Loops Сколько раз выполнялся узел Planning time время построения плана Execution time время выполнения запроса.

Результат представлен на рисунке 4.

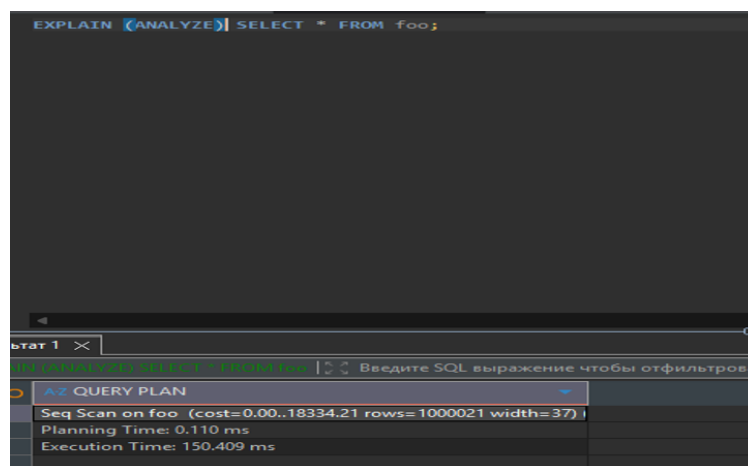


Рис. 4 – План выполнения запроса с анализом

План выполнения с фильтром Rows строки, которые ожидаемо пройдут фильтр Filter: (c1 > 500) условие фильтра

Результат представлен на рисунке 5.

Создали индекс:

```
create index idx_foo_c1 on foo (c1),
```

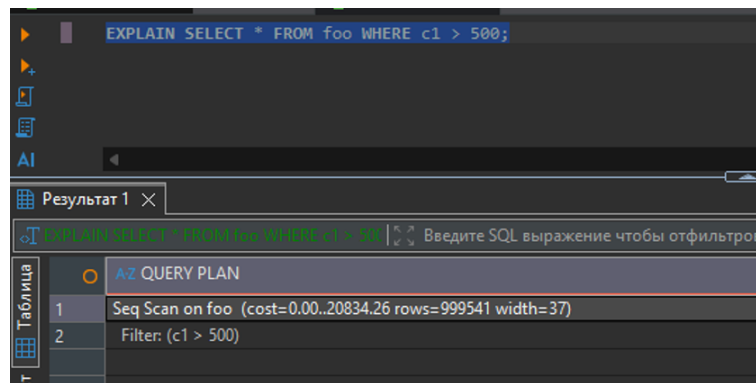


Рис. 5 – План выполнения запроса с анализом

Выполнили:

```
EXPLAIN SELECT * FROM foo WHERE c1 > 500;
```

Ничего не поменялось, условие откидывает много строк, Выполнили:

```
EXPLAIN analyze SELECT * FROM foo WHERE c1 > 500;
```

результат:

```
Seq Scan on foo (cost=0.00..20834.26 rows=999495 width=37) (
- actual time=0.225..149.029 rows=999511 loops=1)
  Filter: (c1 > 500)
    Rows Removed by Filter: 510
    Planning Time: 0.099 ms
    Execution Time: 204.979 ms
```

Индекс будет использоваться при куске таблицы > 20%, потому что это дешевле, при других обстоятельствах, чаще будет использоваться seq scan Вот пример запроса, который будет обрабатываться через индекс:

Результат представлен на рисунке 6.

```
EXPLAIN SELECT * FROM foo WHERE c1 > 1000000;
```

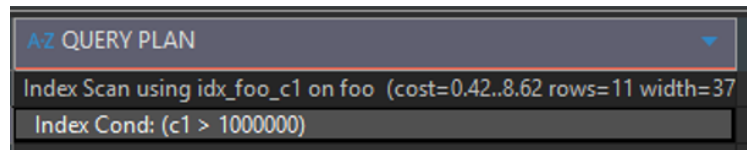


Рис. 6 – План выполнения запроса с фильтром

Выполним запрос:

```
explain SELECT * FROM foo
WHERE c1 < 500 AND c2 LIKE 'abcd%';
```

результат:

```
Index Scan using idx_foo_c1 on foo (cost=0.42..26.93 rows=1 width
— =37)
Index Cond: (c1 < 500)
Filter: (c2 ~~ 'abcd%': :text)
```

2.1

Сначала быстро за счет индекса отбрасываем не нужные строчки, далее ищем совпадения в подстроке. Выполним запрос:

```
explain SELECT * FROM foo
WHERE c2 LIKE 'abcd%';
Gather (cost=1000.00..14552.44 rows=100 width=37)
Workers Planned: 2
-> Parallel Seq Scan on foo (cost=0.00..13542.44 rows=42 width
— =37)
Filter: (c2 ~~ 'abcd%': :text)
```

Нет индекса поэтому последовательно создадим индекс для c2:

```
create index idx_foo_c2 on foo (c2 text_pattern_ops)
```

Выполним запрос:

```
explain SELECT * FROM foo WHERE c2 LIKE 'abcd%';
```

Результат:

```
Index Scan using idx_foo_c2 on foo (cost=0.42..8.45 rows=100 width=37)
Index Cond: ((c2 ~>=~ 'abcd'::text) AND (c2 ~<~ 'abce'::text))
Filter : (c2 ~~ 'abcd%'::text)
```

Теперь мы наглядно видим что используется индекс для работы с текстовыми паттернами

2.2 Удаление индекса c1

выполним команду:

```
drop index idx_foo_c1
```

результат:

```
Gather Merge (cost=63790.40..161021.35 rows=833350 width=37) (actual
— time=111.264..276.359 rows=1000021 loops=1)
Workers Planned: 2
Workers Launched: 2
-> Sort (cost=62790.37..63832.06 rows=416675 width=37) (actual time
— =100.106..135.931 rows=333340 loops=3)
Sort Key: c1
Sort Method: external merge Disk: 16640kB
Worker 0: Sort Method: external merge Disk: 14408kB
Worker 1: Sort Method: external merge Disk: 15136kB
```

```
-> Parallel Seq Scan on foo (cost=0.00..12500.75 rows=416675 width=37)
    — (actual time=0.039..20.431 rows=333340 loops=3)
Planning Time: 0.735 ms
Execution Time: 310.805 ms
```

Прочитает всю таблицу, отсортирует все строки из памяти или диска

2.3 Увеличим объем памяти до 200мб

выполним команду:

```
set_work_mem to "200"MB
```

2.4 Повторим запрос

```
Sort (cost=117994.30..120494.35 rows=1000021 width=37) (actual time
    — =310.384..343.105 rows=1000021 loops=1)
Sort Key: c1
Sort Method: quicksort Memory: 102703kB
-> Seq Scan on foo (cost=0.00..18334.21 rows=1000021 width=37) (actual
    — time=0.035..53.149 rows=1000021 loops=1)
Planning Time: 0.196 ms
Execution Time: 382.072 ms
```

Теперь сортировка выполняется в озу

2.5 Создание индекса для c1

Создадим индекс для поля c1.

```
CREATE INDEX idx_foo_c1 ON foo (c1);
```

Повторим запрос:

```
EXPLAIN ANALYZE SELECT * FROM foo ORDER BY c1;
```

Результат:

```
Index Scan using idx_foo_c1 on foo
(cost=0.42..34317.74 rows=1000021 width=37)
(actual time=1.912..190.837 rows=1000021 loops=1)
Planning Time: 0.685 ms
Execution Time: 220.796 ms
```

Данные читаются в уже отсортированном виде за счет индекса.

2.6 Создание таблицы bar

Создадим таблицу bar:

```
CREATE TABLE bar (
    c1 integer,
    c2 boolean
);
```

Заполним таблицу данными:

```
INSERT INTO bar
SELECT i, i % 2 = 1
FROM generate_series(1, 500000) AS i;
```

2.7 JOIN без индекса

Выполним соединение таблиц без индекса:

```
EXPLAIN ANALYZE
SELECT * FROM foo JOIN bar ON foo.c1 = bar.c1;
```

Результат:

```
Hash Join
(cost=13463.00..40547.29 rows=500000 width=42)
(actual time=100.039..506.343 rows=500010 loops=1)
Hash Cond: (foo.c1 = bar.c1)
-> Seq Scan on foo
(cost=0.00..18334.21 rows=1000021 width=37)
(actual time=0.006..52.525 rows=1000021 loops=1)
-> Hash
(cost=7213.00..7213.00 rows=500000 width=5)
(actual time=99.728..99.730 rows=500000 loops=1)
Buckets: 524288 Batches: 1 Memory Usage: 22163kB
-> Seq Scan on bar
(cost=0.00..7213.00 rows=500000 width=5)
(actual time=0.007..24.519 rows=500000 loops=1)
Planning Time: 0.393 ms
Execution Time: 520.569 ms
```

В данном случае PostgreSQL использует Hash Join.

Сначала читается меньшая таблица `bar`, после чего по ней строится хеш-таблица в памяти. Затем читается большая таблица `foo`. Для каждой строки таблицы `foo` выполняется поиск совпадения в хеш-таблице по полю `c1`.

2.8 JOIN с индексом

Создадим индекс для поля `c1` таблицы `bar`:

```
CREATE INDEX idx_bar_c1 ON bar (c1);
```

Повторим запрос из предыдущего пункта:

```
EXPLAIN ANALYZE SELECT * FROM foo  
JOIN bar ON foo.c1 = bar.c1;
```

Получим следующий план выполнения:

```
Merge Join  
  (cost=2.46..40019.70 rows=500000 width=42)  
  (actual time=0.118..481.829 rows=500010 loops=1)  
    Merge Cond: (foo.c1 = bar.c1)  
-> Index Scan using idx_foo_c1 on foo  
    (cost=0.42..34317.74 rows=1000021 width=37)  
    (actual time=0.030..68.980 rows=500011 loops=1)  
-> Index Scan using idx_bar_c1 on bar  
    (cost=0.42..15212.42 rows=500000 width=5)  
    (actual time=0.016..290.617 rows=500010 loops=1)  
Planning Time: 0.686 ms  
Execution Time: 498.625 ms
```

Теперь PostgreSQL читает таблицу `foo` уже в отсортированном порядке с помощью индекса `idx_foo_c1`.

Одновременно таблица `bar` читается в отсортированном порядке с помощью индекса `idx_bar_c1`.

Поскольку оба потока отсортированы по полю `c1`, PostgreSQL сливает их как два отсортированных списка. В результате используется алгоритм `Merge Join`.

2.9 Очистка кэша

Для проверки влияния кэша необходимо очистить кэш операционной системы. Для этого были выполнены следующие команды:

```
sudo service postgresql stop
sudo sync
sudo su root
echo 3 > /proc/sys/vm/drop_caches
exit
sudo service postgresql start
sudo -u admin psql lab2
```

После этого выполним запрос:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM foo;
```

Полученные результаты можно представить следующим образом:

Запуск	shared_hit	shared_read	Комментарий
1-й	≈ 0	много	Данные читаются с диска
2-й	растёт	уменьшается	Часть данных уже в кэше PostgreSQL
3-й	почти всё	почти 0	Почти всё берётся из shared buffers

`shared_hit` — блоки, полученные из кэша PostgreSQL.

`shared_read` — блоки, прочитанные с диска.

Таким образом, повторное выполнение запроса позволяет увидеть влияние кэширования данных на работу PostgreSQL.

3 Эффективные методы индексации в PostgreSQL

Создадим таблицу `users` и добавим необходимые индексы. После этого заполним таблицу данными.

3.1 Индекс для поиска по фрагменту текста `ILIKE`

Выполним запрос:

```
EXPLAIN
SELECT count ( * )
FROM users
WHERE email ILIKE '%yandex.ru%';
```

До создания индекса PostgreSQL использует последовательное сканирование таблицы:

```
Finalize Aggregate (cost=16924.12..16924.13 rows=1 width=8)
-> Gather (cost=16923.90..16924.11 rows=2 width=8)
Workers Planned: 2
-> Partial Aggregate (cost=15923.90..15923.91 rows=1 width=8)
-> Parallel Seq Scan on users (cost=0.00..15650.33 rows=109428 width=0)
Filter : (email ~~* '%yandex.ru%'::text)
```

Для эффективного поиска по фрагменту текста подключим расширение `pg_trgm`:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Расширение `pg_trgm` добавляет поддержку триграмм — разбиения строки на последовательности из трёх символов.

Создадим GIN-индекс:

```
CREATE INDEX idx_users_email_trgm ON users USING gin (email gin_trgm_ops);
```

После создания индекса повторим запрос.

В результате используется следующая последовательность операций:

```
Finalize Aggregate (cost=15562.98..15562.99 rows=1 width=8) (actual time
— =658.663..671.236 rows=1 loops=1)
-> Gather (cost=15562.77..15562.98 rows=2 width=8) (actual time
— =657.489..671.218 rows=3 loops=1)
Workers Planned: 2
Workers Launched: 2
-> Partial Aggregate (cost=14562.77..14562.78 rows=1 width=8) (actual time
— =594.369..594.371 rows=1 loops=3)
-> Parallel Bitmap Heap Scan on users (cost=2479.35..14289.20 rows
— =109428 width=0) (actual time=134.492..582.651 rows=83239 loops=3)
— Recheck Cond: (email ~~* '%yandex.ru%': :text)
Heap Blocks: exact=3456
-> Bitmap Index Scan on idx_users_email_trgm (cost=0.00..2413.70 rows
— =262626 width=0) (actual time=147.458..147.458 rows=249717 loops
— =1)
Index Cond: (email ~~* '%yandex.ru%': :text)
```

- Bitmap Index Scan — индекс `idx_users_email_trgm` быстро находит все строки, в которых содержится `yandex.ru` без учёта регистра;
- Parallel Bitmap Heap Scan — по найденным идентификаторам строк читаются сами строки из таблицы;
- Recheck Cond — GIN-индекс с `pg_trgm` может давать небольшое количество ложных срабатываний, поэтому PostgreSQL дополнительно проверяет условие;
- Partial Aggregate, Gather и Finalize Aggregate — выполняется параллельный подсчёт `COUNT(*)`.

Таким образом, использование GIN-индекса с `pg_trgm` позволяет эффективно выполнять поиск по части текстового значения.

3.2 Частичный индекс по updated_at только для администраторов

Выполним запрос:

```
EXPLAIN
SELECT *
FROM users
WHERE updated_at = '2016-03-22 01:51:57.261738'::timestamp
AND type = 'admin';
```

До создания индекса PostgreSQL использует последовательное сканирование:

```
Gather  (cost=1000.00..17692.10 rows=1 width=50) (actual time
— =97.721..111.076 rows=0 loops=1)
Workers Planned: 2
Workers Launched: 2-> Parallel Seq Scan on users  (cost=0.00..16692.00 rows
— =1 width=50) (actual time=83.822..83.822 rows=0 loops=3)
Filter : ( (updated_at = '2016-03-22 01:51:57.261738'::timestamp without time zone)
— AND (type = 'admin'::text) )
Rows Removed by Filter: 333333
Planning Time: 0.615 ms
Execution Time: 111.127 ms
```

Создадим частичный индекс, который будет содержать только записи для пользователей с типом admin:

```
CREATE INDEX idx_users_updated_at_admin
ON users (updated_at)
WHERE type = 'admin';
```

После создания индекса результат изменился:

```
Index Scan using idx_users_updated_at_admin on users  (cost=0.29..8.31 rows=1
— width=50) (actual time=0.116..0.117 rows=0 loops=1)
```

Index Cond: (updated_at = '2016-03-22 01:51:57.261738'::timestamp without time zone)
—
Planning Time: 1.214 ms
Execution Time: 0.150 ms

Таким образом, частичный индекс позволяет индексировать только ту часть таблицы, которая соответствует условию `type = 'admin'`.

3.3 Индекс по JSONB-полю extra

Выполним запрос:

```
EXPLAIN
SELECT count(*)
FROM users
WHERE extra @> '{"extra_data": true}':::jsonb;
```

До создания индекса PostgreSQL выполняет последовательное сканирование:

```
Finalize Aggregate (cost=16707.35..16707.36 rows=1 width=8) (actual time
— =159.600..168.944 rows=1 loops=1) -> Gather (cost
— =16707.14..16707.35 rows=2 width=8) (actual time
— =159.319..168.931 rows=3 loops=1)
Workers Planned: 2
Workers Launched: 2 -> Partial Aggregate (cost=15707.14..15707.15 rows=1
— width=8) (actual time=144.850..144.852 rows=1 loops=3) -> Parallel Seq
— Scan on users (cost=0.00..15650.33 rows=22722 width=0) (actual time
— =0.042..142.365 rows=18558 loops=3)
Filter : (extra @> '{"extra_data": true}':::jsonb)
Rows Removed by Filter: 314776
Planning Time: 0.985 ms
Execution Time: 169.090 ms
```

Создадим GIN-индекс для поля `extra`:

```
CREATE INDEX idx_users_extra
ON users USING gin (extra);
```

После создания индекса стоимость выполнения запроса изменилась:

```
Aggregate (cost=12078.63..12078.64 rows=1 width=8) (actual time
— =73.431..73.435 rows=1 loops=1)
-> Bitmap Heap Scan on users (cost=818.63..11942.30 rows=54533 width=0)
— (actual time=17.425..68.542 rows=55673 loops=1)
Recheck Cond: (extra @> '{"extra_data": true}': :jsonb)
Heap Blocks: exact=10399
-> Bitmap Index Scan on idx_users_extra (cost=0.00..805.00 rows=54533
— width=0) (actual time=14.849..14.850 rows=55673 loops=1)
Index Cond: (extra @> '{"extra_data": true}': :jsonb)
Planning Time: 37.276 ms
Execution Time: 73.522 ms
```

GIN-индекс позволяет эффективно выполнять поиск по содержимому поля типа `jsonb`.

3.4 Итог

В ходе работы были рассмотрены различные способы индексации в PostgreSQL:

Задача	Тип индекса	Команда
Поиск по части email	GIN + pg_trgm	USING gin (email gin_trgm_ops)
Только для администраторов	Частичный B-tree	WHERE type = 'admin'
Поиск по jsonb	GIN	USING gin (extra)

В результате были рассмотрены различные планы выполнения запросов PostgreSQL, влияние индексов на выбор планировщика, работу Hash Join и Merge Join, а также влияние памяти и кэширования на выполнение запросов.